

# 基于 Java Swing 的学生成绩管理系统设计与实现

## 一、项目简介

设计一个图形化学生成绩管理系统。用户可以通过 Swing 窗口完成学生信息录入、显示、查询、修改、删除、成绩统计、排序和文件保存读取等操作。

系统采用文件作为数据存储方式，不使用数据库。学生信息保存在本地文本文件中，程序启动后可以读取文件，程序运行过程中可以保存数据。

## 二、系统功能设计

### 1. 主界面功能

主界面可以采用 JFrame，整体布局如下（注意，这里只是一个示意，具体排版请自行安排）：

-----

| 学生成绩管理系统 |

-----

| 学号: [     ] 姓名: [     ] |

| 年龄: [     ] |

| Java 成绩: [     ] |

| 数学成绩: [     ] |

| 英语成绩: [     ] |

-----

| [添加] [修改] [删除] [查询] [显示全部] [排序] |

| [保存到文件] [从文件读取] [清空输入框] |

-----

| JTable 表格显示学生信息 |

| 学号 | 姓名 | 年龄 | Java | 数学 | 英语 | 总分 | 平均分 | 等级 |

-----

## 三、核心功能要求

### 1. 添加学生

用户在文本框中输入学生信息，点击“添加”按钮，将学生加入系统。

基本要求：

学号不能为空  
姓名不能为空  
年龄必须为整数  
成绩必须是数字  
成绩范围为 0 到 100  
学号不能重复

## 2. 显示所有学生

使用 JTable 显示所有学生信息。

表格列包括：

学号  
姓名  
年龄  
Java 成绩  
数学成绩  
英语成绩  
总分  
平均分  
等级

## 3. 查询学生

用户输入学号，点击“查询”按钮。

如果找到学生：

将学生信息显示在输入框中  
同时可以弹出提示框显示查询成功

如果没有找到：

弹出提示框：未找到该学生

## 4. 修改学生信息

用户先输入或查询学号，再修改成绩或基本信息，点击“修改”按钮。

修改后刷新表格。

## 5. 删除学生

用户输入学号，点击“删除”按钮。

删除前可以弹出确认框：

`JOptionPane.showConfirmDialog(...)`

## 6. 成绩计算

系统自动计算：

总分

平均分

等级

等级规则：

平均分  $\geq 90$ ：优秀

平均分  $\geq 80$ ：良好

平均分  $\geq 70$ ：中等

平均分  $\geq 60$ ：及格

平均分  $< 60$ ：不及格

## 7. 排序功能

点击“按平均分排序”按钮后，按照平均分从高到低排序，并刷新表格。

## 8. 文件保存与读取

使用 IO 流将数据保存到文件：

`students.txt`

文件格式：

2024001,张三,19,88,92,85

2024002,李四,20,76,81,79

2024003,王五,19,95,90,93

读取时重新生成学生对象，并显示到表格中。

# 四、建议类设计

## 1. ScoreCalculator 接口

体现接口的使用。

```
public interface ScoreCalculator {
    double getTotalScore();

    double getAverageScore();

    String getLevel();
}
```

## 2. Student 类

表示学生对象。

```
public class Student implements ScoreCalculator {
    private String id;
    private String name;
    private int age;
    private double javaScore;
    private double mathScore;
    private double englishScore;

    public Student(String id, String name, int age,
                   double javaScore, double mathScore, double
englishScore) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.javaScore = javaScore;
        this.mathScore = mathScore;
        this.englishScore = englishScore;
    }

    @Override
    public double getTotalScore() {
        return javaScore + mathScore + englishScore;
    }

    @Override
    public double getAverageScore() {
        return getTotalScore() / 3;
    }

    @Override
    public String getLevel() {
        double avg = getAverageScore();

        if (avg >= 90) {
            return "优秀";
        } else if (avg >= 80) {
            return "良好";
        } else if (avg >= 70) {
            return "中等";
        } else if (avg >= 60) {
            return "及格";
        } else {

```

```

        return "不及格";
    }
}

// getter 和 setter 省略
}

```

### 3. ExcellentStudent 子类

用于体现继承。

```

public class ExcellentStudent extends Student {
    private String reward;

    public ExcellentStudent(String id, String name, int age,
                           double javaScore, double mathScore,
double englishScore,
                           String reward) {
        super(id, name, age, javaScore, mathScore, englishScore);
        this.reward = reward;
    }

    public String getReward() {
        return reward;
    }

    public void setReward(String reward) {
        this.reward = reward;
    }
}

```

在项目中可以这样使用：

如果学生平均分大于等于 90，则可以认为该学生是优秀学生。

也可以不单独录入优秀学生，只在讲解继承时作为扩展类使用。

### 4. StudentManager 类

负责学生数据管理。

```

import java.util.ArrayList;
import java.util.Comparator;

public class StudentManager {
    private ArrayList<Student> students = new ArrayList<>();

    public boolean addStudent(Student student) {
        if (findStudentById(student.getId()) != null) {
            return false;
        }
        students.add(student);
        return true;
    }
}

```

```

    public Student findStudentById(String id) {
        for (Student student : students) {
            if (student.getId().equals(id)) {
                return student;
            }
        }
        return null;
    }

    public boolean deleteStudentById(String id) {
        Student student = findStudentById(id);
        if (student == null) {
            return false;
        }
        students.remove(student);
        return true;
    }

    public boolean updateStudent(Student newStudent) {
        Student oldStudent = findStudentById(newStudent.getId());
        if (oldStudent == null) {
            return false;
        }

        oldStudent.setName(newStudent.getName());
        oldStudent.setAge(newStudent.getAge());
        oldStudent.setJavaScore(newStudent.getJavaScore());
        oldStudent.setMathScore(newStudent.getMathScore());
        oldStudent.setEnglishScore(newStudent.getEnglishScore());

        return true;
    }

    public void sortByAverageScore() {
        students.sort(new Comparator<Student>() {
            @Override
            public int compare(Student s1, Student s2) {
                return Double.compare(s2.getAverageScore(),
s1.getAverageScore());
            }
        });
    }

    public ArrayList<Student> getStudents() {
        return students;
    }

    public void setStudents(ArrayList<Student> students) {
        this.students = students;
    }
}

```

## 5. FileUtil 类

负责文件保存和读取。

```

import java.io.*;
import java.util.ArrayList;

public class FileUtil {
    public static void saveStudents(ArrayList<Student> students,
String fileName) {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(fileName))) {
            for (Student s : students) {
                writer.write(s.getId() + "," +
                    s.getName() + "," +
                    s.getAge() + "," +
                    s.getJavaScore() + "," +
                    s.getMathScore() + "," +
                    s.getEnglishScore());
                writer.newLine();
            }
        } catch (IOException e) {
            throw new RuntimeException("保存文件失败: " +
e.getMessage());
        }
    }

    public static ArrayList<Student> readStudents(String
fileName) {
        ArrayList<Student> students = new ArrayList<>();

        File file = new File(fileName);
        if (!file.exists()) {
            return students;
        }

        try (BufferedReader reader = new BufferedReader(new
FileReader(fileName))) {
            String line;

            while ((line = reader.readLine()) != null) {
                String[] arr = line.split(",");

                if (arr.length != 6) {
                    continue;
                }

                String id = arr[0];
                String name = arr[1];
                int age = Integer.parseInt(arr[2]);
                double javaScore = Double.parseDouble(arr[3]);
                double mathScore = Double.parseDouble(arr[4]);
                double englishScore = Double.parseDouble(arr[5]);

                Student student = new Student(id, name, age,
javaScore, mathScore, englishScore);
                students.add(student);
            }
        } catch (IOException e) {
            throw new RuntimeException("读取文件失败: " +

```

```

        e.getMessage());
    }

    return students;
}
}

```

## 五、Swing 界面类设计

### 1. StudentFrame 类

该类负责图形界面。

主要组件包括：

```

JFrame
JLabel
JTextField
JButton
JTable
JScrollPane
JPanel
JOptionPane
DefaultTableModel

```

建议界面类名称：

```

public class StudentFrame extends JFrame {
}

```

核心成员变量：

```

private JTextField idField;
private JTextField nameField;
private JTextField ageField;
private JTextField javaScoreField;
private JTextField mathScoreField;
private JTextField englishScoreField;

private JTable table;
private DefaultTableModel tableModel;

private StudentManager manager = new StudentManager();

```

表格列名：

```

String[] columnNames = {
    "学号", "姓名", "年龄", "Java 成绩", "数学成绩", "英语成绩", "总分",
    "平均分", "等级"
};

```

刷新表格的方法：

```

private void refreshTable() {
    tableModel.setRowCount(0);

    for (Student s : manager.getStudents()) {

```



```

        Object[] rowData = {
            s.getId(),
            s.getName(),
            s.getAge(),
            s.getJavaScore(),
            s.getMathScore(),
            s.getEnglishScore(),
            s.getTotalScore(),
            String.format("%.2f", s.getAverageScore()),
            s.getLevel()
        };

        tableModel.addRow(rowData);
    }
}

```

从输入框创建学生对象：

```

private Student getStudentFromInput() {
    String id = idField.getText().trim();
    String name = nameField.getText().trim();
    int age = Integer.parseInt(ageField.getText().trim());
    double javaScore =
        Double.parseDouble(javaScoreField.getText().trim());
    double mathScore =
        Double.parseDouble(mathScoreField.getText().trim());
    double englishScore =
        Double.parseDouble(englishScoreField.getText().trim());

    return new Student(id, name, age, javaScore, mathScore,
        englishScore);
}

```

清空输入框：

```

private void clearInput() {
    idField.setText("");
    nameField.setText("");
    ageField.setText("");
    javaScoreField.setText("");
    mathScoreField.setText("");
    englishScoreField.setText("");
}

```

## 六、Swing 需要补充给学生的资料

请见 PPT。

## 七、提交材料

请以“学号\_姓名\_Java 实践项目.zip”命名压缩包，并提交以下内容（把这些内容都放在压缩包里）：

#### 1. 源代码文件

包括 Main.java、Student.java、ExcellentStudent.java、ScoreCalculator.java、StudentManager.java、FileUtil.java、StudentFrame.java 等完整代码文件。

#### 2. 数据文件

提交 students.txt，文件中至少包含 50 条学生测试数据，且能够被程序正常读取。

#### 3. 项目说明文档

文档中应包括项目名称、学生信息、功能介绍、类的设计说明、使用到的 Java 知识点、文件读写说明、运行结果截图和项目总结。

#### 4. 运行截图

至少提交 5 张截图，包括主界面、添加学生、查询学生、修改学生、删除学生、保存或读取文件等功能截图（注意：截图自行压缩，整个 zip 大小不要超过 10M）。

提交链接为：

<https://send2me.cn/HBis2ydq/SN-UQZoErY9VKQ>